Practical 1: Introduction to Components of IoT

Aim: To study and understand the basic components of an Internet of Things (IoT) system using Tinkercad simulation.

Apparatus Required: Arduino Uno, Breadboard(Optional), LED, Resistor, Connecting Wires, Sensors (Optional), Computer with Tinkercad.

Theory:

The **Internet of Things (IoT)** refers to a network of physical devices that are embedded with sensors, software, and communication capabilities to exchange data over the internet. It plays a major role in automation and smart systems such as smart homes, healthcare, and industrial automation (SCADA systems).

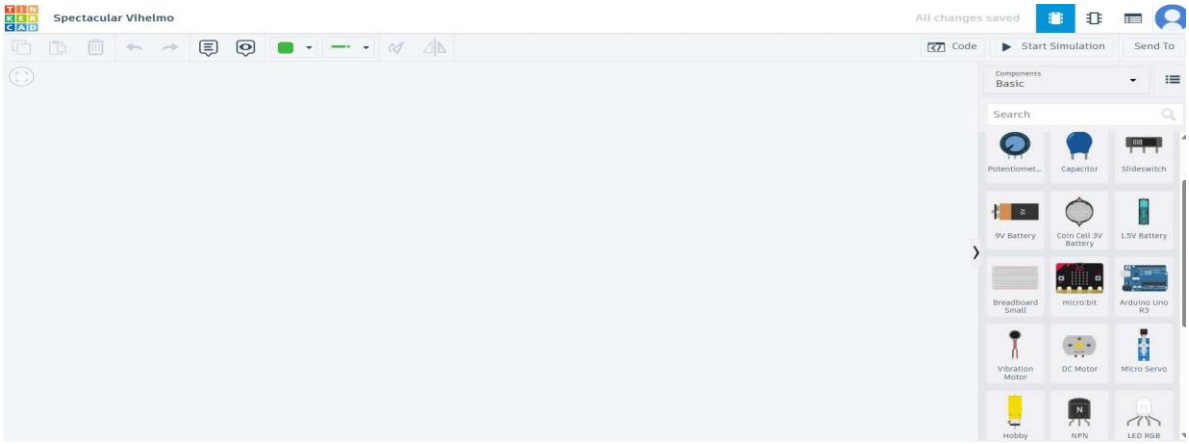
The main components of IoT include:

1. **Sensors:**
   Sensors are used to collect data from the environment such as temperature, humidity, light, and motion. These inputs are essential for monitoring systems.
2. **Microcontroller (Arduino):**
   The Arduino Uno acts as the brain of the system. It processes input data and controls output devices based on programmed instructions.
3. **Actuators:**
   These are output devices such as LEDs, motors, and buzzers that perform actions based on commands from the microcontroller.
4. **Communication Module:**
   It enables data transfer between devices and networks (Wi-Fi, Bluetooth, etc.).
5. **Software Platform:**
   Platforms like Tinkercad simulate real-world circuits and allow testing without physical hardware.

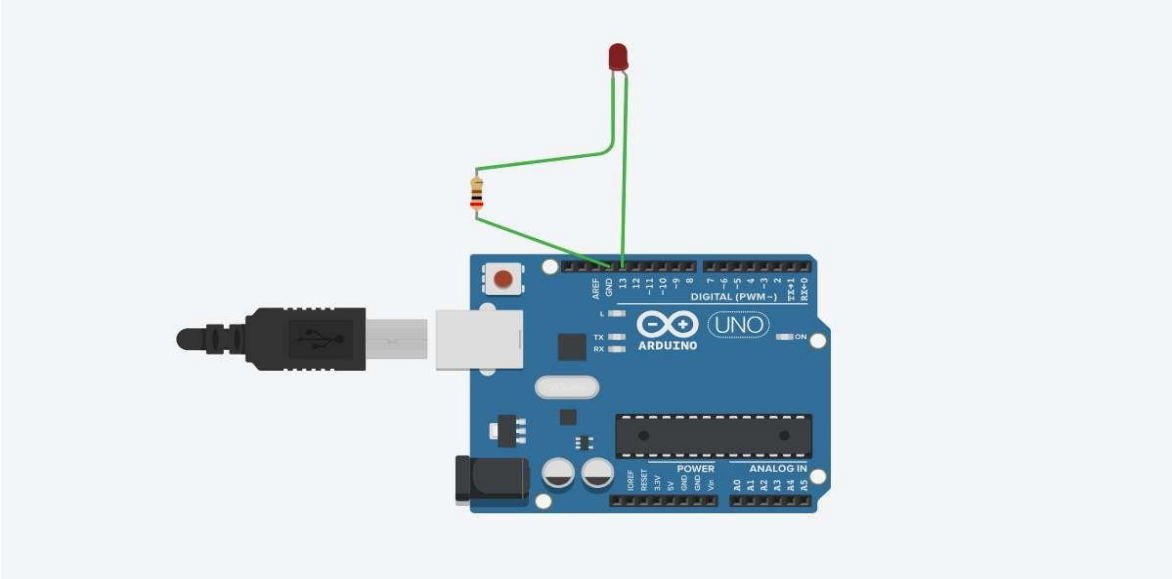
IoT systems are widely used in **SCADA systems** for remote monitoring and control of industrial processes.

Procedure:

1. Open Tinkercad Click on “Create New Circuit”.



2. Add Arduino Uno and other components like LED and resistor.



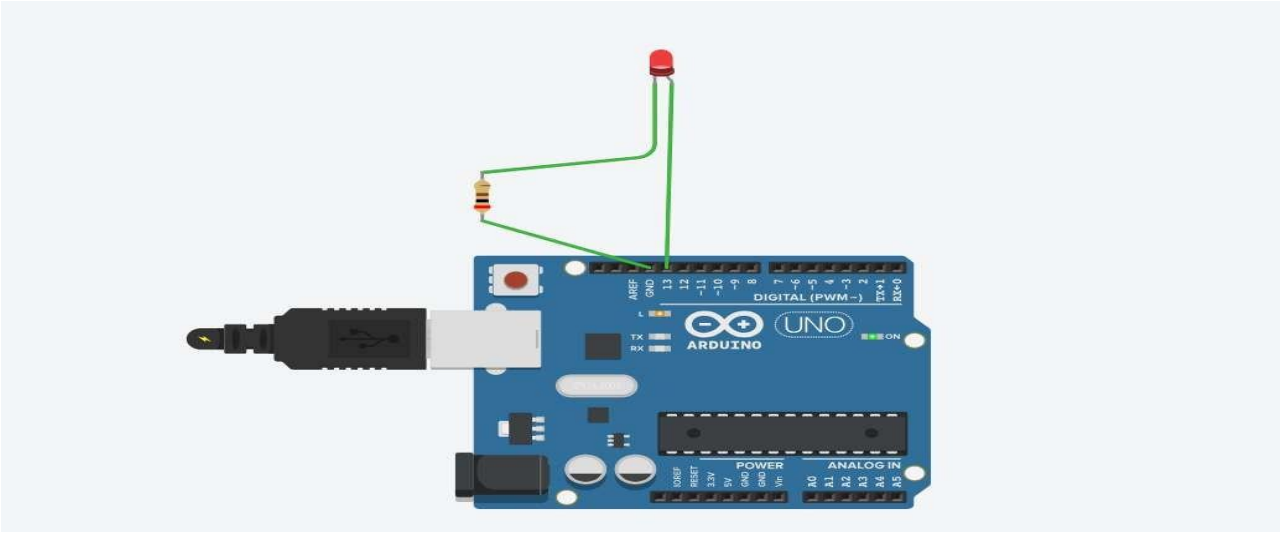
3. Run simulation and analyze behavior.

Observation:

- Learned how Arduino interacts with input and output devices.
- Understood basic IoT architecture and working.

Result:

Successfully studied and understood the components and working of IoT systems using simulation.



Practical 2: Blinking LED

Aim: To design and simulate a circuit to blink an LED using Arduino.

Apparatus Required: Arduino Uno, LED, 220Ω Resistor, Connecting Wires

Theory: An LED (Light Emitting Diode) is a semiconductor device that emits light when current flows through it. In this experiment, the Arduino is programmed to send HIGH and LOW signals to the LED, causing it to turn ON and OFF repeatedly.

- **HIGH (1):** LED ON
- **LOW (0):** LED OFF

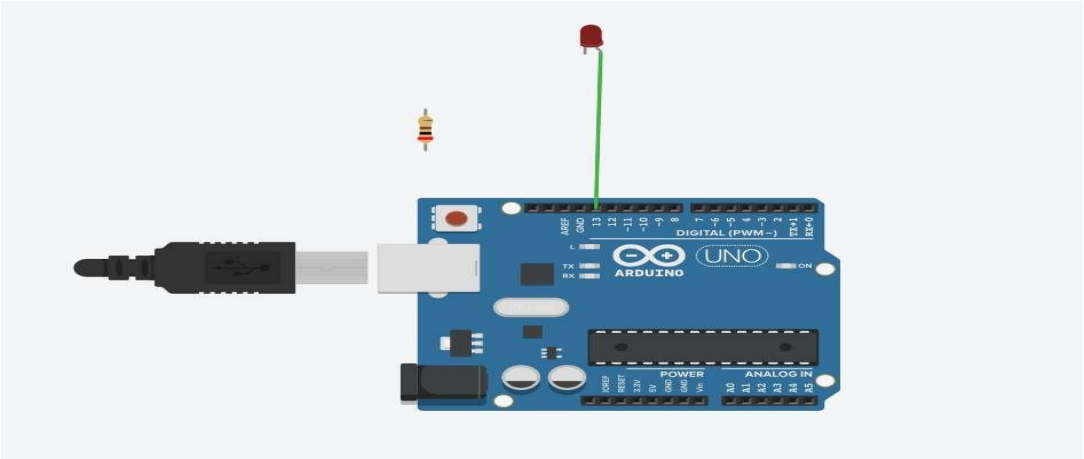
The delay function controls the time interval between ON and OFF states, resulting in a blinking effect.

Circuit Diagram:

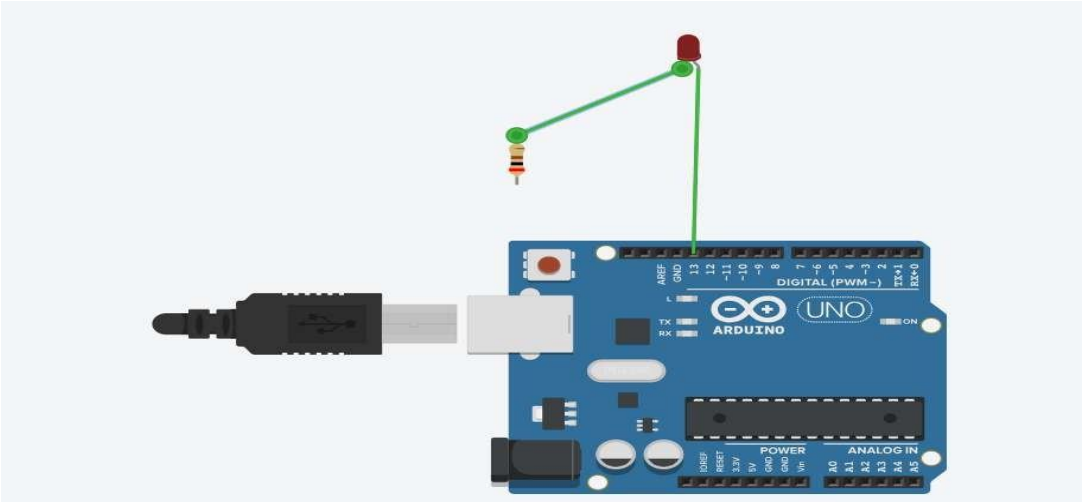
- LED anode → Pin 13
- LED cathode → Resistor → Ground

Procedure:

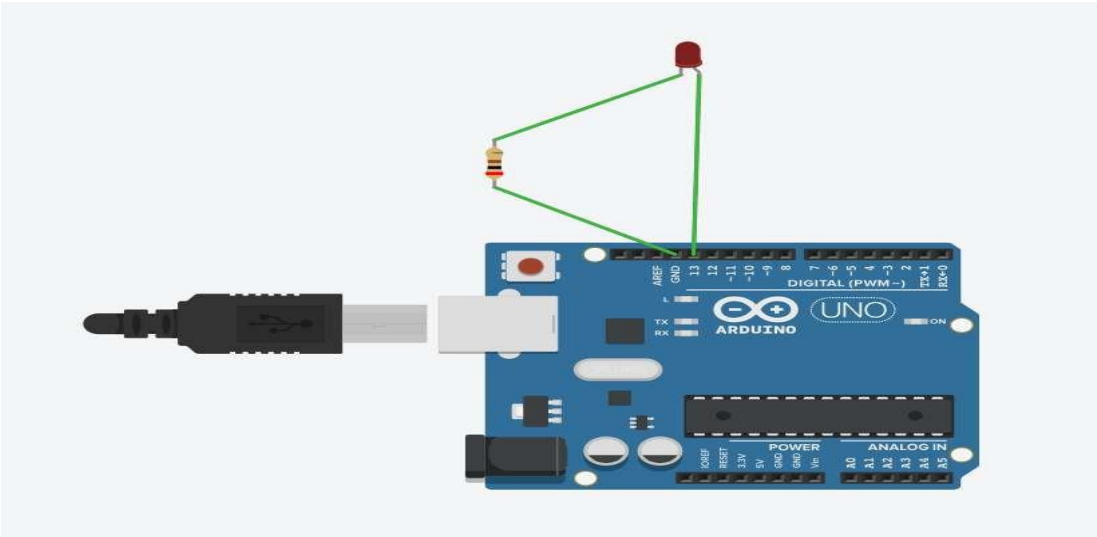
1. Connect the LED to Arduino pin 13.



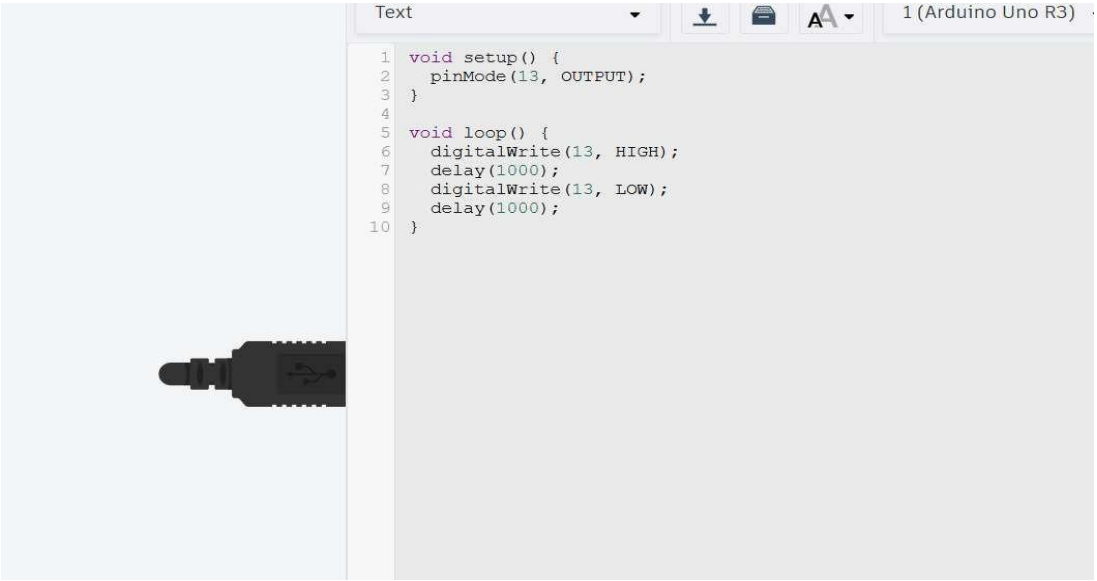
2. Connect a resistor in series to avoid excess current.



3. Connect the circuit to ground.



4. Write and upload the code.



Code: -

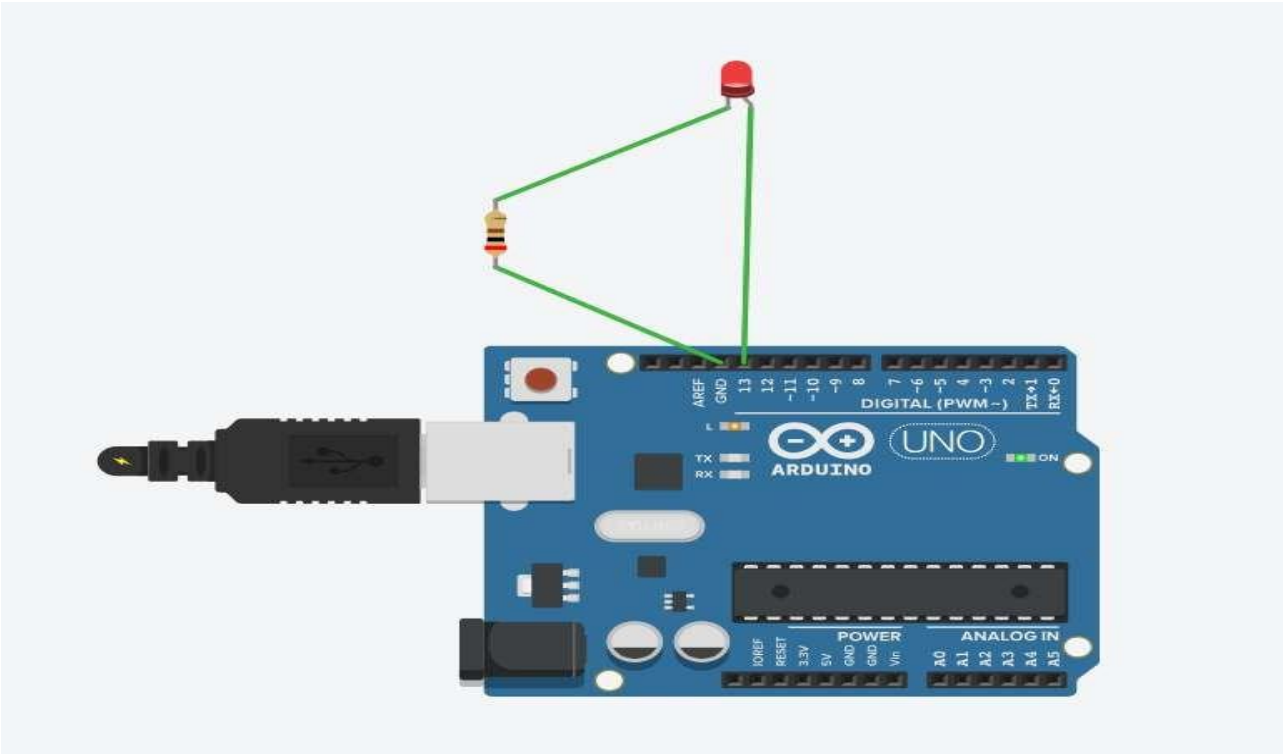
```
void setup() {
  pinMode(13, OUTPUT);
}

void loop() {
  digitalWrite(13, HIGH);
  delay(1000);
  digitalWrite(13, LOW);
  delay(1000);
}
```

Observation:

- LED turns ON for 1 second and OFF for 1 second continuously.

Result:

The LED blinking circuit was successfully implemented and verified.



Practical 3: Interfacing LED and Switch

Aim: To control an LED using a push button switch.

Apparatus Required: Arduino Uno, LED, Push Button, Resistor, Wires

Theory:

A push button acts as an input device. When pressed, it sends a signal (HIGH) to the Arduino, which then activates the LED. This demonstrates **digital input-output interfacing**, which is a basic concept in embedded systems.

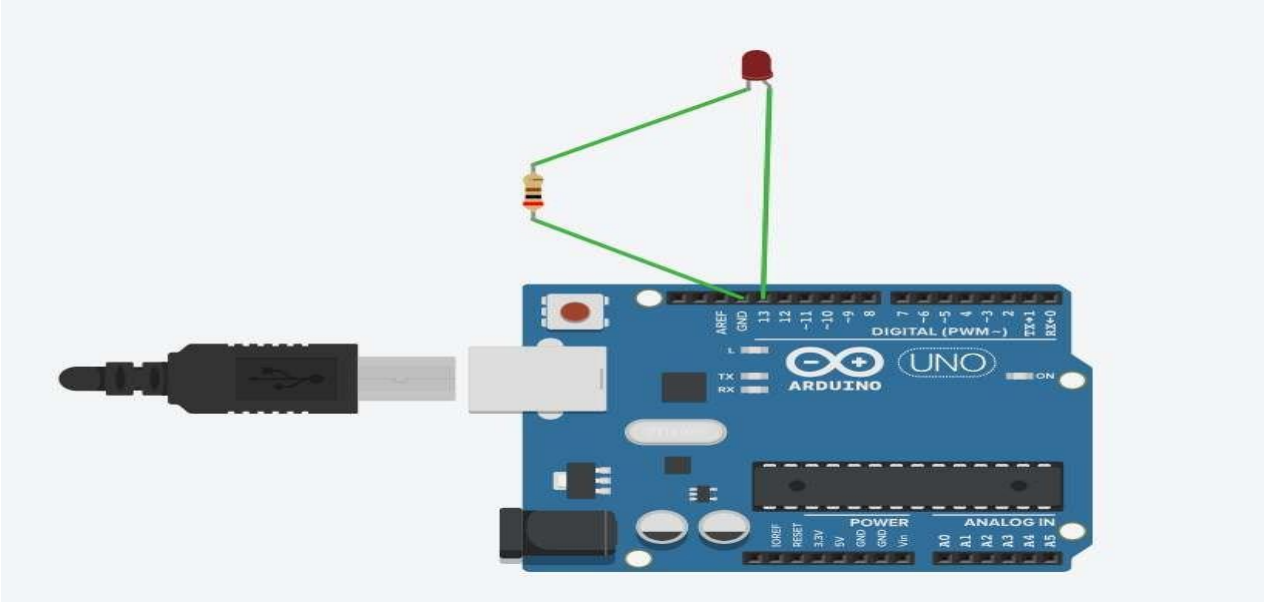
- Button Pressed → HIGH → LED ON
- Button Released → LOW → LED OFF

Circuit Diagram:

- Button → Pin 2
- LED → Pin 13

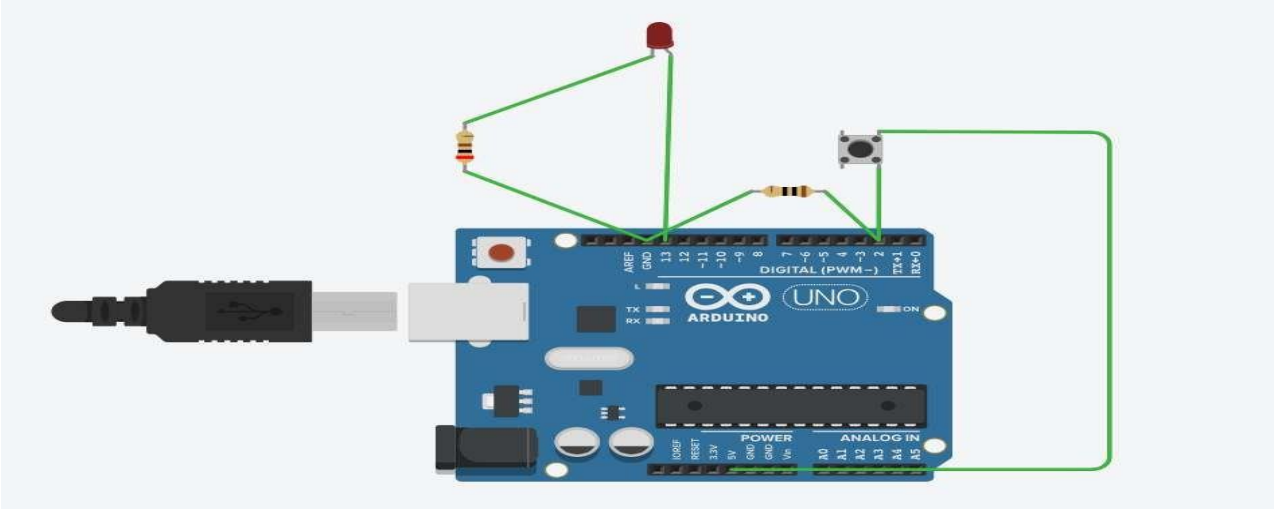
Procedure:

1. LED Connection:
    - Connect LED Anode (long leg) → Pin 13 (Arduino)
    - Connect LED Cathode (short leg) → 220Ω Resistor
    - Connect other end of resistor → GND



2. Push Button Connection:
   - Connect one side of Push Button → 5V (Arduino)
   - Connect other side → Pin 2 (Arduino)

3. Pull-Down Resistor Connection:
   - Connect 10kΩ Resistor between:
     - Pin 2 (same button pin) → GND



Connection Explanation

In this circuit, the push button is used as an **input device** and the LED acts as an **output device**.
When the button is pressed, a HIGH signal (5V) is sent to **pin 2** of the Arduino. The Arduino reads this input and sends a HIGH signal to **pin 13**, turning ON the LED.
When the button is not pressed, the pull-down resistor ensures that the input remains LOW, preventing false triggering due to noise.
This experiment demonstrates **digital input-output interfacing**, which is widely used in IoT and SCADA systems for controlling devices based on user input.

Code: -

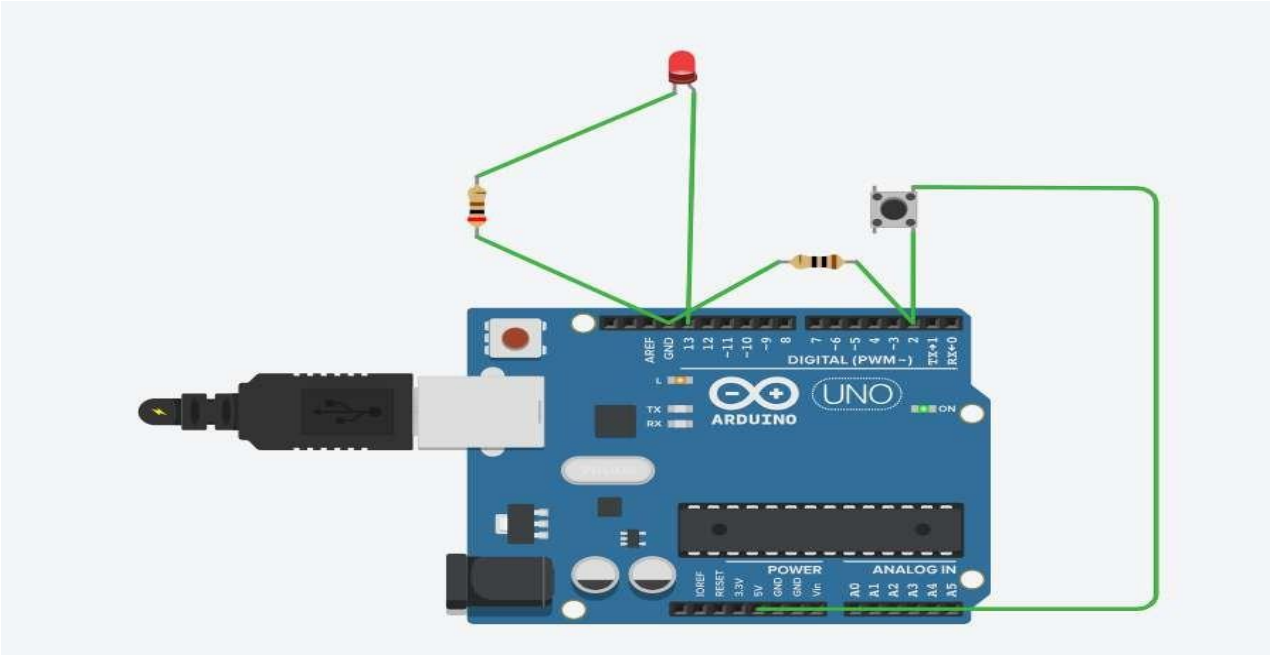
```
int button = 2;
int led = 13;

void setup() {
  pinMode(button, INPUT);
  pinMode(led, OUTPUT);
}

void loop() {
  int state = digitalRead(button);
  if(state == HIGH) {
  digitalWrite(led, HIGH);
  } else {
    digitalWrite(led,  LOW);
  }
}
```

Observation:

- LED glows only when the button is pressed.

Result:

Successfully controlled LED using a switch.



Practical 4: Stepper Motor Connection

Aim: To interface and control a stepper motor using Arduino.

Apparatus Required: Arduino Uno, Stepper Motor, Motor Driver (ULN2003), Wires

Theory:

A **stepper motor** is an electromechanical device that converts electrical pulses into discrete mechanical movements (steps). Unlike DC motors, it does not rotate continuously but moves in fixed angles, making it suitable for applications requiring precise control such as robotics, CNC machines, and industrial automation systems.

Since the Arduino cannot provide sufficient current to drive a motor directly, a **motor driver IC (L293D)** is used. The L293D is a **dual H-bridge motor driver**, which allows control of the direction and operation of motors.

Working Principle:

- The Arduino sends digital signals to the input pins of the L293D.
- The L293D amplifies these signals and drives the motor.
- By controlling the sequence of inputs, the motor rotates step-by-step.
- The enable pin activates the motor driver section.

Advantages of Stepper Motor:

- High precision control
- No need for feedback system
- Reliable and easy to control
- Widely used in automation and SCADA systems

Procedure:

1. **Arduino to L293D:**

   - Pin 8 → Input 1 (Pin 2 of L293D)
   - Pin 9 → Input 2 (Pin 7 of L293D)
   - Pin 10 → Input 3 (Pin 10 of L293D) *(if using stepper)*
   - Pin 11 → Input 4 (Pin 15 of L293D) *(if using stepper)*

2. **Motor Connection:**

   - Motor terminals → Output pins of L293D
     - Output 1 (Pin 3) → Motor
     - Output 2 (Pin 6) → Motor

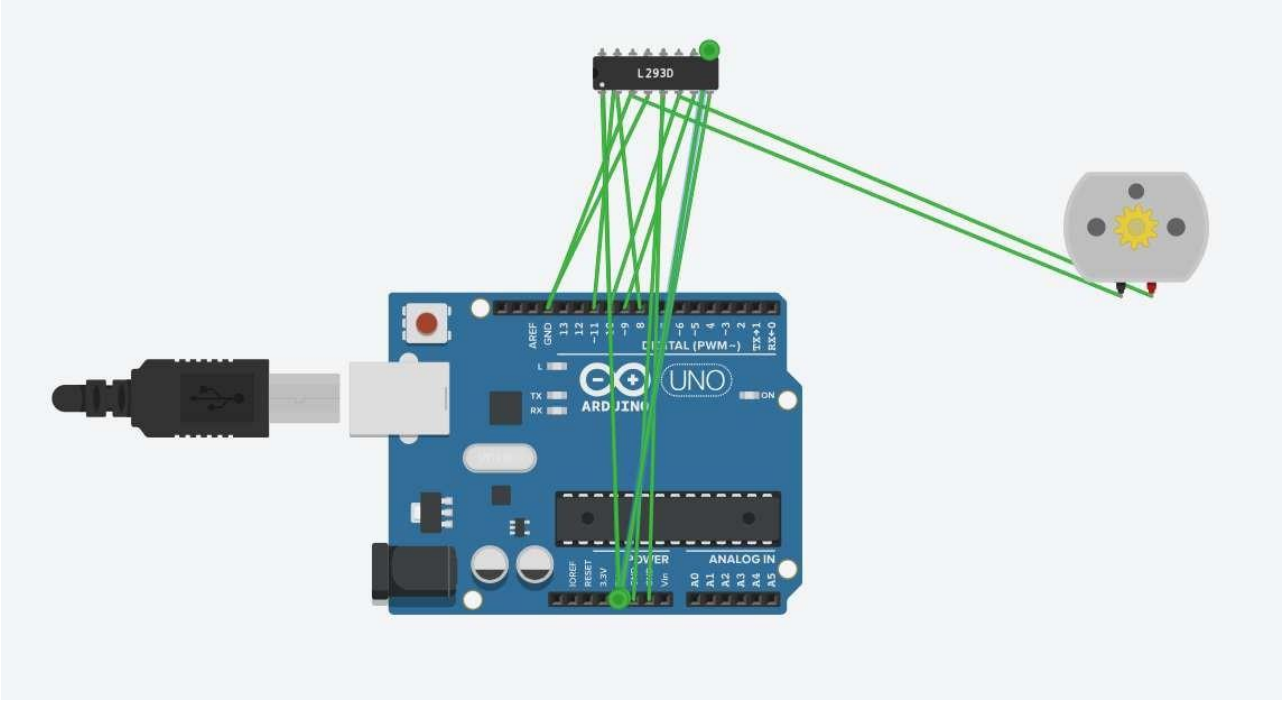
   (For stepper, multiple outputs are used)

3. **Power Connections:**

   - Pin 16 (Vcc1) → 5V (Arduino)
   - Pin 8 (Vcc2) → 5V (Motor Supply)
   - Pins 4, 5, 12, 13 → GND

4. Enable Pins:

- Pin 1 (Enable 1) → 5V
- Pin 9 (Enable 2) → 5V



Code: -

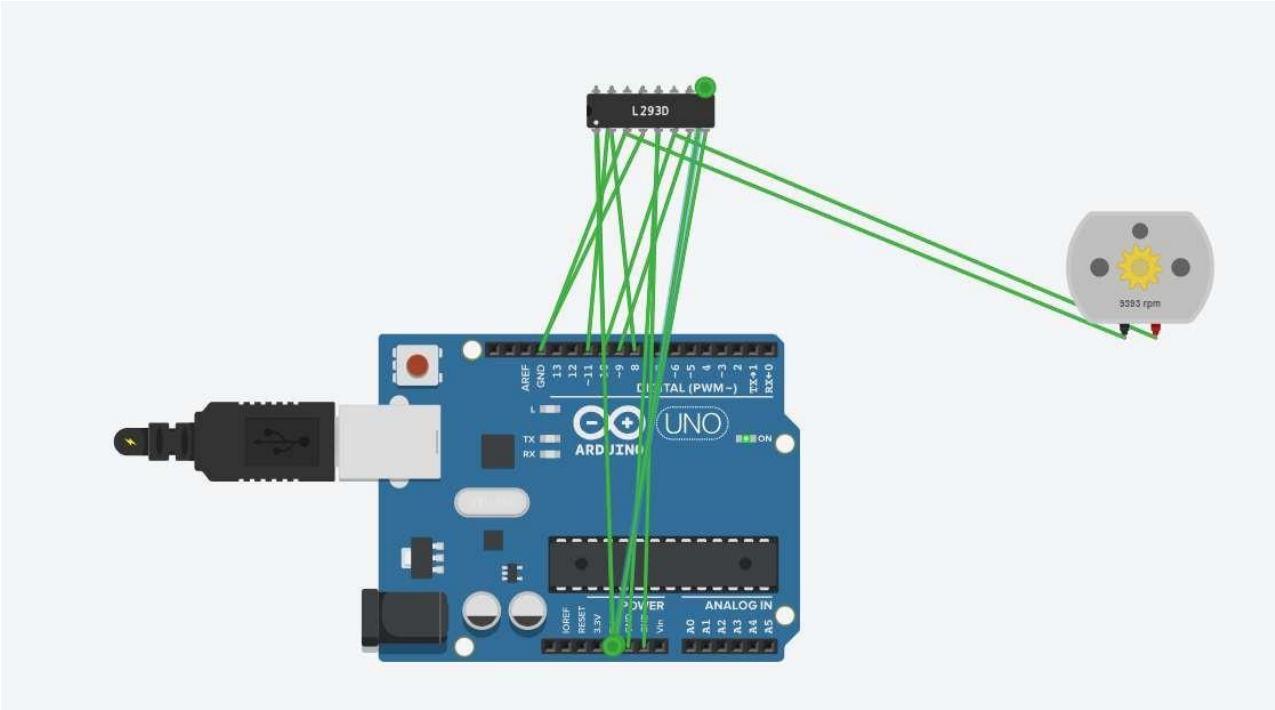
```
int motorPin1 = 8;
int motorPin2 = 9;

void setup() {
  pinMode(motorPin1, OUTPUT);
  pinMode(motorPin2,  OUTPUT);
}

void loop() {
  digitalWrite(motorPin1, HIGH);
  digitalWrite(motorPin2, LOW);
  delay(2000);

  digitalWrite(motorPin1,   LOW);
  digitalWrite(motorPin2,  HIGH);
  delay(2000);
}
```

Observation:

- The motor rotates in one direction for a few seconds.
- Then it changes direction based on input signals.
- Smooth operation is observed through the motor driver.

Result:

The stepper motor was successfully interfaced and controlled using Arduino and L293D motor driver. The motor responded correctly to input signals and demonstrated controlled rotation.



Practical 5: Simulation of Traffic Signal

Aim: To simulate a traffic signal system using LEDs and Arduino.

Apparatus Required: Arduino Uno, Red LED, Yellow LED, Green LED, Resistors, Wires

Theory:

A **traffic signal system** is used to control the movement of vehicles and pedestrians at road intersections.
It ensures smooth traffic flow and reduces accidents by providing clear visual signals.
The system operates using three colored lights:

- **Red Light:** Indicates STOP
- **Yellow Light:** Indicates WAIT or READY
- **Green Light:** Indicates GO

In this experiment, the Arduino is programmed to control three LEDs in a specific time sequence, simulating a real-world traffic signal system.

Working Principle:

- The Arduino sends HIGH and LOW signals to the LEDs.
- Each LED is turned ON and OFF in a fixed time interval using delay functions.
- The sequence followed is:
    - Red → Yellow → Green → Repeat

This type of automation is widely used in **SCADA systems**, smart cities, and intelligent transportation systems.

Connections:

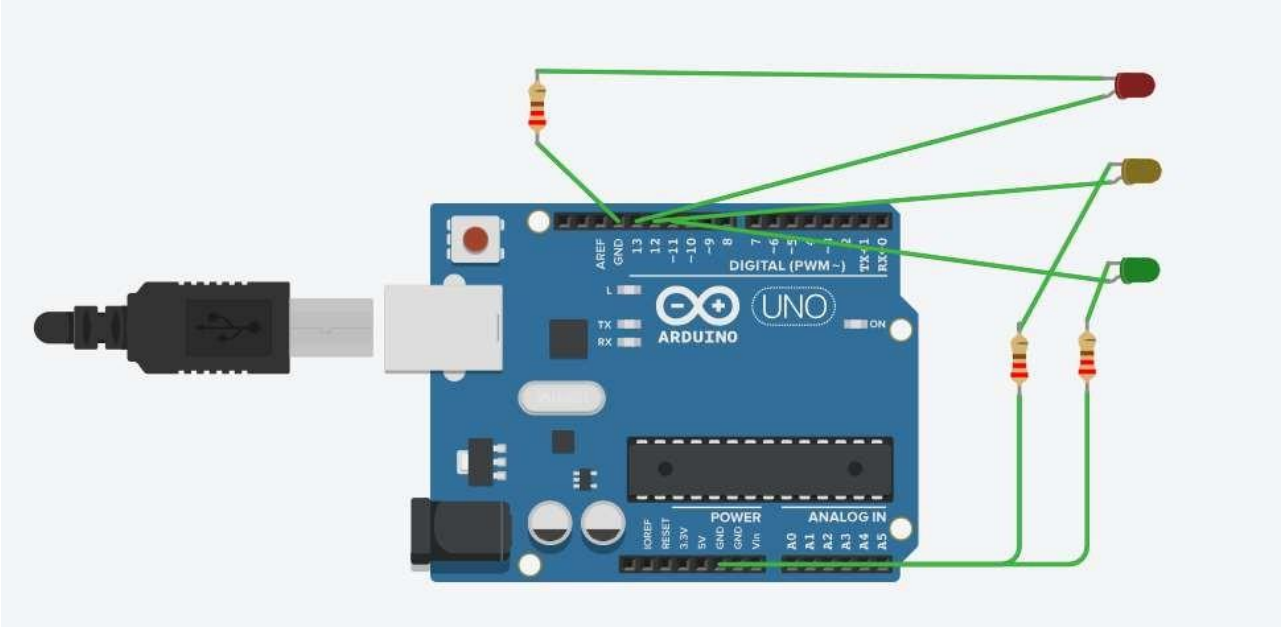
1. Red LED:
- Anode → **Pin 13 (Arduino)**
- Cathode → **220Ω Resistor → GND**

2. Yellow LED:
- Anode → **Pin 12 (Arduino)**
- Cathode → **220Ω Resistor → GND**

3. Green LED:
- Anode → **Pin 11 (Arduino)**
- Cathode → **220Ω Resistor → GND**

Connection Explanation

Each LED is connected to a separate digital pin of the Arduino. The Arduino controls the LEDs by sending HIGH (ON) and LOW (OFF) signals. Resistors are used to limit the current and protect the LEDs from damage.
The sequential activation of LEDs represents the real-time functioning of a traffic signal system.



Procedure:

1. Open Tinkercad and create a new circuit.
2. Place Arduino Uno and three LEDs on the breadboard.
3. Connect each LED to different digital pins (11, 12, 13).
4. Add resistors in series with each LED.
5. Connect all grounds properly.
6. Write the Arduino program.
7. Start simulation and observe LED sequence.

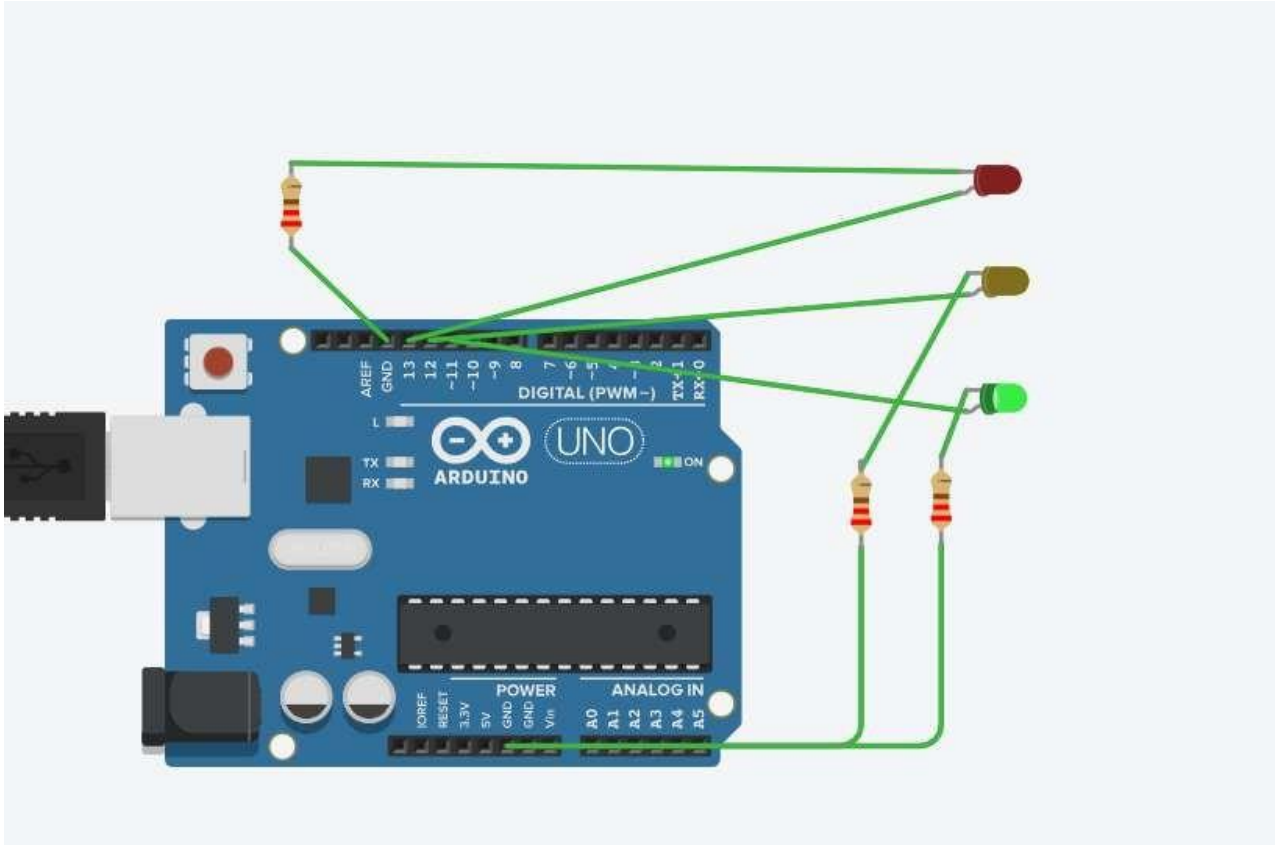
Code:-

```
int red = 13;
int yellow = 12;
int green = 11;

void setup() {
pinMode(red, OUTPUT);
pinMode(yellow, OUTPUT);
pinMode(green, OUTPUT);
}

void loop() {
// Red Light ON
digitalWrite(red, HIGH);
delay(5000);
digitalWrite(red, LOW);

// Yellow Light ON
digitalWrite(yellow, HIGH);
delay(2000);
digitalWrite(yellow, LOW);
```

```
// Green Light ON
digitalWrite(green, HIGH);
delay(5000);
digitalWrite(green, LOW);
}
```

Observation:

- Red LED glows first, indicating stop.
- Yellow LED glows next, indicating wait.
- Green LED glows last, indicating go.
- The sequence repeats continuously.

Result:

The traffic signal system was successfully simulated using Arduino. The LEDs operated in the correct sequence, demonstrating real-world traffic signal behavior.